

IoT-Based Smart Elderly Safety Band: A Wearable Emergency Monitoring and Caregiver Notification Prototype

Kalahe Gamage Nilupul Ruvinda, Kailavakesan Tharmeeagan, Piruththikaa Sivarajah

* The authors are with the Department of Engineering, Faculty of Engineering, University of Sri Jayewardenepura, Sri Lanka.

† This project was carried out as part of CO3302 Computer Engineering Project. Manuscript submitted September 27, 2026.

Abstract—Elderly individuals living independently face risks from falls, prolonged inactivity, and sudden emergencies during which a conventional smartphone may not be quickly accessible. This paper presents a compact, low-cost wearable elderly safety-band prototype that performs local emergency monitoring and provides caregiver notification through Bluetooth Low Energy (BLE). The system integrates motion sensing via a 3-axis accelerometer/gyroscope (MPU6050), manual panic activation, inactivity monitoring, local alerts (OLED display, buzzer, and LEDs), a warning/cancellation state machine, and a Flutter-based caregiver mobile application. An ESP32-C3 microcontroller performs sensing and decision-making locally, while BLE provides an additional short-range caregiver notification path. Fall-like motion is detected using a temporal pattern of downward movement, movement duration, and sudden stop/impact rather than a single acceleration sample, and a nominal 1-meter downward-movement requirement is approximated through motion characteristics rather than directly measured. The prototype was evaluated through controlled bench and hand-held demonstrations covering panic activation, inactivity detection, sleep mode, and fall-like motion, with a staged warning-cancellation-emergency response used to reduce false alerts. The system is presented as a university engineering prototype rather than a certified medical device, and implemented features are clearly distinguished from proposed future work such as Wi-Fi/cellular long-range notification and calibrated battery-voltage sensing.

Index Terms—Wearable systems, elderly monitoring, fall detection, Internet of Things, ESP32, MPU6050, Bluetooth Low Energy, caregiver notification, embedded systems.

I. INTRODUCTION

ELDERLY individuals who live alone or spend extended periods unsupervised face a particular safety risk: a fall, a sudden medical event, or a period of prolonged inactivity may leave them unable to reach or operate a conventional smartphone to request help. Delayed assistance in such situations can significantly worsen outcomes. Commercial smartwatches and general-purpose wearables address a broad range of lifestyle and fitness functions, but they are not always designed around the specific, focused requirements of elderly emergency response.

This project presents the IoT-Based Smart Elderly Safety Band, a wearable prototype that performs first-level emergency sensing and local response directly on the device,

and additionally notifies a caregiver through a companion mobile application over Bluetooth Low Energy (BLE). Rather than attempting to reproduce the full feature set of a commercial smartwatch, the system is intentionally scoped around a small number of safety-relevant functions: fall-like motion detection, manual panic activation, inactivity monitoring, local alerting, and caregiver notification.

The remainder of this paper is organized as follows. Section II states the problem and objectives. Section III gives a system overview. Section IV describes the hardware design and component justification. Section V details the fall detection and emergency decision logic. Section VI describes the BLE communication protocol and the Flutter caregiver application. Section VII summarizes the software implementation. Section VIII presents the testing and demonstration procedure. Section IX discusses the results. Sections X and XI outline limitations and future work, and Section XII concludes the paper.

II. PROBLEM STATEMENT AND OBJECTIVES

A. Problem Statement

An elderly person may fall, remain inactive for an extended period, or experience a sudden emergency, and may be unable to reach, unlock, or operate a smartphone quickly enough to call for help. A dedicated wearable device that senses these situations directly on the body and can respond locally — independent of whether a caregiver happens to be looking at a phone — reduces this gap in response time.

B. Objectives

The objectives of this project are to:

- 1) Develop a compact wearable prototype built around an ESP32-C3 microcontroller.
- 2) Monitor user movement using an MPU6050 inertial measurement unit (IMU).
- 3) Detect fall-like movement using a sequence of motion conditions rather than a single acceleration reading.
- 4) Provide a manual panic button for user-initiated emergency requests.
- 5) Detect prolonged inactivity that may indicate an unresponsive user.
- 6) Provide a warning stage with a cancellation opportunity before escalating to emergency, in order to reduce false alerts.

- 7) Provide local feedback to the user and bystanders through an OLED display, a buzzer, and status LEDs.
- 8) Transmit warning and emergency information to a Flutter-based mobile application over BLE for caregiver notification.
- 9) Operate from a rechargeable LiPo battery, independent of a laptop USB connection, to support true wearable/portable use.

III. SYSTEM OVERVIEW

The system follows a Sense → Process → Decide → Local Alert → BLE Notification → Caregiver App pipeline, illustrated conceptually in Fig. 1.

The MPU6050 and two push-buttons (panic and sleep/cancel) provide raw input to the ESP32-C3, which is the central controller of the system. The ESP32-C3 continuously processes sensor and button data, executes the fall-detection, inactivity-detection, panic, sleep, warning, and emergency logic, and drives the local output devices (OLED, buzzer, and RGB status LEDs). In parallel, the ESP32-C3 advertises status and alert information over BLE to a Flutter caregiver application.

A key architectural decision in this project is that *all emergency decision-making is performed locally on the ESP32-C3*. The mobile application is an additional caregiver notification and monitoring interface, not the primary emergency-detection mechanism. Even if no phone is connected, the wearable can still activate its local buzzer, LEDs, and OLED display to alert the user and any nearby bystanders.

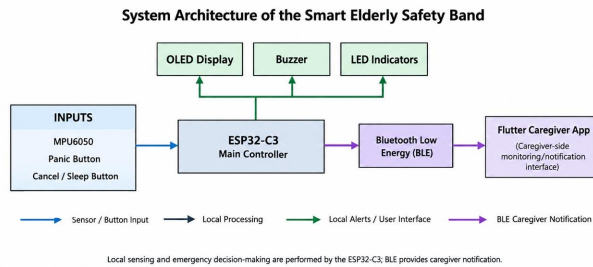


Fig. 1. System architecture: the MPU6050 and buttons feed the ESP32-C3, which drives local alerts (OLED, buzzer, LEDs) and sends caregiver notifications to the Flutter application over BLE.

The system operates in one of three primary states — Normal, Warning, and Emergency — with an additional Sleep Mode that selectively suppresses inactivity monitoring. These states are described in detail in Section V.

IV. HARDWARE DESIGN

The hardware was selected to balance processing capability, wireless connectivity, power consumption, size, and cost, consistent with a wearable prototype.

A. ESP32-C3 Super Mini

The ESP32-C3 Super Mini serves as the main controller of the system. It reads the MPU6050 and button inputs, executes the fall, inactivity, panic, sleep, warning, and emergency logic, drives the OLED, buzzer, and LEDs, and communicates with the Flutter application through its built-in BLE radio. The ESP32-C3 was chosen over alternative platforms for the following reasons:

- Arduino Uno: lacks built-in BLE and is physically larger, requiring an additional Bluetooth module.
- Raspberry Pi: higher cost and power consumption than required for this real-time sensor/control task.
- ESP8266: provides Wi-Fi but not the built-in BLE capability required by this design.

The ESP32-C3 combines sufficient processing capability, compact size, low power consumption, adequate GPIO, and integrated BLE, reducing the need for a separate wireless module.

B. MPU6050 Inertial Measurement Unit

The MPU6050 provides 3-axis accelerometer and 3-axis gyroscope readings, used to observe motion, acceleration changes, downward movement, sudden impact, and inactivity. The gyroscope additionally provides rotational/orientation information used to tolerate reasonable tilt during motion analysis (Section V).

Camera-based and ultrasonic/time-of-flight (ToF) sensing were considered and rejected for this task: a camera raises privacy concerns, requires greater processing power, and must maintain coverage of the user, which is impractical for a body-worn device; ultrasonic/ToF sensors measure distance along a fixed sensor axis, which is unreliable when the sensor's orientation changes freely with the wearer's body and arm position. An IMU-based, motion-pattern approach is better suited to arbitrary wearable orientation.

C. OLED Display

A 0.96-inch I2C OLED (SSD1306, 128×64) provides a local visual interface, showing normal, warning, emergency, sleep, and battery status. This assists both the user and, during demonstration, an observer in understanding the current device state.

D. Buzzer and LEDs

A TMB12A05 active buzzer provides an immediate, local audible warning, allowing nearby people to become aware of an emergency without needing to check the mobile application. Three LEDs provide a simple visual state indication: green for normal operation, blue for sleep mode, and red for warning/emergency conditions.

E. Buttons

A panic button allows the user to manually request help at any time, independent of the automatic detection logic, which cannot be guaranteed to cover every possible emergency scenario. A separate sleep/cancel button serves two purposes: cancelling a false warning before it escalates to an emergency, and toggling sleep mode, which prevents inactivity monitoring from generating false alerts while the user is intentionally resting.

F. Power Subsystem

The prototype is powered by a rechargeable LiPo battery, enabling portable, wearable operation independent of a laptop USB connection. The power path is:

LiPo battery → *TP4056 (charging/protection)* → *slide switch*
→ *MT3608 (boost)* → *ESP32-C3 5V input*

The TP4056 module provides USB-based charging and protection for the LiPo cell. The MT3608 boost converter raises the battery voltage to approximately 5V for the ESP32-C3's 5V input path; its output must be adjusted to the required voltage prior to connection. Once powered from the battery, the prototype operates independently of a laptop, which is used only during development and firmware programming.

The current battery-level indication shown on the OLED is a runtime-based estimate rather than a directly measured battery voltage, and is reported as such rather than as an accurately measured percentage. A calibrated implementation would require a voltage-divider circuit, use of the ESP32 ADC, and calibration against the specific battery discharge curve; this is discussed further as future work in Section XI.

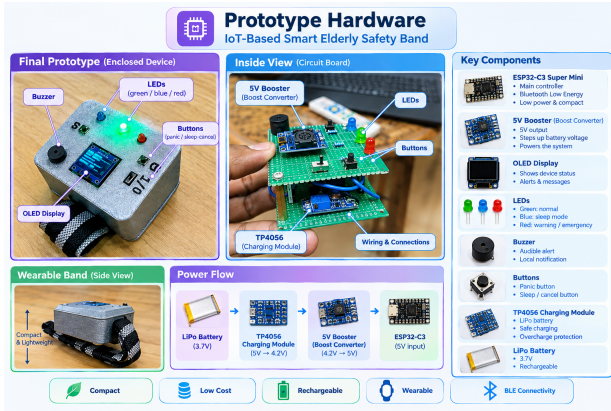


Fig. 2. Assembled prototype hardware. Top left: enclosed final device showing the OLED display, buzzer, panic/sleep buttons, and status LEDs. Top right: internal circuit view showing the ESP32-C3, MT3608 boost converter, and TP4056 charging module. Bottom: wearable band form factor and the LiPo-to-ESP32-C3 power path.

Fig. 2 shows the assembled prototype. The enclosed device integrates the OLED display, buzzer, panic button, sleep/cancel button, and the green/blue/red status LEDs on a single compact enclosure worn on a wrist band, with the

ESP32-C3, MT3608 boost converter, and TP4056 charging module mounted internally.

TABLE I ESP32-C3 Pin Assignments

Function	GPIO / Pin
OLED / MPU6050 SDA (I2C)	GPIO4
OLED / MPU6050 SCL (I2C)	GPIO5
OLED / MPU6050 VCC	3.3 V
Buzzer	GPIO6
Red LED	GPIO3
Green LED	GPIO10
Blue LED	GPIO7
Panic button (INPUT_PULLUP, other side to GND)	GPIO1
Sleep/Cancel button (INPUT_PULLUP, other side to GND)	GPIO0

V. FALL DETECTION AND EMERGENCY LOGIC

Fall detection is the most technically demanding aspect of the system, since the MPU6050 provides only acceleration and angular-velocity data and cannot directly measure a physical fall height.

A. Motion Sensing and the 1-Meter Limitation

The MPU6050 cannot directly determine that an object has moved exactly 1.00 m; it measures only acceleration and angular velocity. Consequently, the nominal 1-meter downward-movement requirement used in this project is *approximated* rather than directly measured, using a combination of downward-motion detection, acceleration characteristics, movement duration, and an estimated movement quantity derived from the accelerometer signal, together with detection of a sudden stop or impact. This is an important and deliberate distinction that is preserved throughout this paper: the system does not claim to measure fall height directly.

B. Fall-Like Motion Sequence

The fall detector is designed to avoid relying on any single acceleration sample. Instead, it evaluates a temporal sequence:

fast downward movement + *sufficient movement/duration*
+ *sudden stop/impact* = *possible fall*

C. Impact Confirmation and Rejection of Shake/Tilt

Two failure modes were identified during testing and addressed in the detection logic:

Shake false triggering. Continuous shaking can generate acceleration spikes, movement changes, and impact-like peaks that may accidentally satisfy parts of the fall sequence in isolation. The detector therefore requires a valid

downward-movement sequence prior to accepting an impact as fall confirmation; a shake alone, without a preceding valid downward-movement phase, does not confirm a fall.

Tilt tolerance. Tilting the device alone is not treated as a fall. Because the hand cannot move in a perfectly vertical line during the hand-held demonstration, a reasonable degree of tilt is tolerated during an otherwise valid downward-movement phase, without making tilt rejection so strict that genuine fall-like motion is missed.

Held-device vs. free-fall dynamics. During testing, free-fall-style drops were detected more reliably than fast downward hand movements, because a device released into free fall experiences an acceleration profile that approaches zero, whereas a device held in the hand is supported by the hand and therefore experiences a more complex, noisier acceleration signal. For this reason, the final detection logic relies on the downward-movement pattern, movement duration, acceleration change, and impact/sudden-stop conditions rather than requiring pure free-fall behavior.

TABLE II Key Fall-Detection Constants (as configured during testing)

Constant	Value
FREE_FALL_THRESHOLD	7500.0
IMPACT_THRESHOLD	24000.0
MOVEMENT_THRESHOLD	1000.0
DOWNWARD_START_DROP	3000.0
DOWNWARD_TOTAL_ACCEL_MAX	22000.0
DOWNWARD_MOVEMENT_THRESHOLD	2500.0
REQUIRED_DOWNWARD_DISTANCE_M	1.00
MIN_DOWNWARD_TRAVEL_TIME	250 ms
DOWNWARD_DISTANCE_GAIN	1.40
DOWNWARD_ACCEL_NOISE_MPS2	0.80
FREE_FALL_MIN_TIME	260 ms
POST_FALL_CONFIRM_TIME	0 ms
POST_FALL_CHECK_WINDOW	2500 ms
FALL_COOLDOWN	5000 ms
IMPACT_AFTER_DOWNWARD_GRACE_TIME	450 ms
MIN_VALID_DOWNWARD_SAMPLES	2
DOWNWARD_CONTINUE_DROP	800.0
DOWNWARD_CONTINUE_MOVE	800.0

D. Warning and Cancellation

When a fall-like sequence, a panic-button press, or an inactivity timeout is detected, the system enters the Warning state rather than immediately declaring an emergency. This provides a cancellation opportunity intended to reduce false emergency alerts. If the user cancels the warning (via the sleep/cancel button) within the configured window, the system returns to Normal; otherwise, it escalates to Emergency, as summarized in Fig. 3.

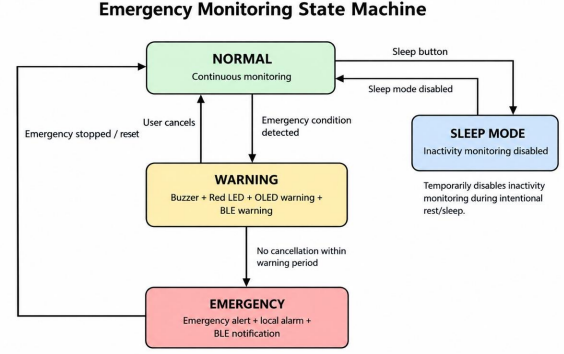


Fig. 3. Emergency monitoring state machine. From Normal, a detected condition (panic, fall-like motion, or inactivity) moves the system to Warning, which either returns to Normal if the user cancels or escalates to Emergency if not cancelled within the warning period. Sleep Mode is a side state entered/exited from Normal that suppresses inactivity monitoring only.

E. Inactivity Detection

The system monitors ongoing movement and raises an inactivity warning if no significant movement is observed for a configured timeout. For demonstration purposes this timeout was set to a short interval (e.g., approximately 15 seconds); for realistic deployment, a substantially longer period would be configured.

F. Panic Button

Pressing the panic button transitions the system directly to the Warning state, independent of the fall-detection logic, followed by the same cancellation opportunity and escalation path as other trigger types.

G. Sleep Mode

Sleep mode is provided because an elderly user may remain still for extended periods while intentionally resting or sleeping, which would otherwise be indistinguishable from an inactivity emergency. Enabling sleep mode suppresses inactivity monitoring only; it does not disable the entire safety system, and the panic and fall-detection pathways remain active according to the firmware configuration.

VI. BLE COMMUNICATION AND FLUTTER APPLICATION

The ESP32-C3 advertises as a BLE peripheral named *ElderlyBand*, using a custom GATT service (UUID 12345678-1234-1234-1234-1234567890ab) with an alert characteristic (UUID abcd1234-5678-90ab-cdef-1234567890ab) used to transmit status and alert messages, including: DEVICE_STARTED; BATTERY / BATTERY_EST; WARNING:/EMERGENCY:/CANCELLED: variants for PANIC BUTTON, PANIC FALL ALERT, and INACTIVITY; STOPPED:EMERGENCY; and SLEEP_MODE_ON/SLEEP_MODE_OFF.

The companion mobile application is implemented in Dart using the Flutter framework and functions primarily as a caregiver monitoring interface. It connects to the wearable over BLE and displays device connection status, battery/status information, and warning/emergency events as they occur. The Flutter application is explicitly not the primary fall-detection processor; all detection and decision-making logic executes locally on the ESP32-C3, and the application serves as an additional caregiver notification and monitoring channel rather than the sole emergency mechanism.



Fig. 4. Flutter caregiver application. Left: home screen in the Normal state with the band disconnected. Middle: device connection and battery panel, with connection controls and mode/event summary. Right: Emergency alert state following a detected fall, showing the alert banner and the corresponding entry in the alert history.

BLE was selected over Wi-Fi or GSM/cellular connectivity for this prototype because it is low-power, is already integrated into the ESP32-C3, and does not require a router or cellular subscription for direct device-to-phone communication. This choice comes with a known limitation: BLE is short-range, and if a caregiver is outside BLE range, a direct BLE notification will not reach them. Addressing this limitation with Wi-Fi/cloud or GSM/cellular connectivity is identified as future work (Section XI) and is not claimed as a current capability of the prototype.

VII. SOFTWARE IMPLEMENTATION

The ESP32-C3 firmware is implemented in Arduino/C++-style embedded code and performs periodic sensor acquisition, button-state handling, the Normal/Warning/Emergency/Sleep state machine, fall-detection and inactivity-detection logic, local output control (OLED, buzzer, LEDs), and BLE message transmission. The Flutter caregiver application handles BLE connection management, parsing of incoming status/alert messages, and presentation of device state to the caregiver.

VIII. TESTING AND DEMONSTRATION

The prototype was evaluated through a series of controlled bench and hand-held tests covering each of the

system's trigger types. The following acceptance criteria were used to guide testing:

- Slow and normal movements should remain in the Normal state.
- Fast movement that does not satisfy the full fall sequence should be rejected.
- A small shake alone should not confirm a fall.
- Tilt alone should not confirm a fall.
- A valid fall-like downward movement followed by a sudden stop/impact should produce a Warning promptly.
- The user should be able to cancel a Warning before it escalates.
- A non-cancelled Warning should escalate to Emergency.
- Panic-button activation should generate a Warning independently of motion state.
- Inactivity should generate a Warning after the configured timeout, unless Sleep Mode is active.
- The Flutter application should receive BLE status/alert messages within communication range.

For the fall-detection demonstration specifically, the device is held still during an initial calibration period, then moved quickly downward by hand through approximately one meter, with the device remaining in the user's grasp throughout. Because the motion is hand-held rather than a released object, slight tilt and a small residual shake at the end of the motion are expected and tolerated by the detection logic. This procedure is described throughout as a *controlled fall-like motion test*, and is explicitly distinguished from an exact, physically measured 1-meter free-fall test.

The live demonstration sequence exercises, in order: normal monitoring, Flutter BLE connection, panic-button activation, warning cancellation, sleep mode, inactivity detection, fall-like motion detection, and emergency escalation.

IX. RESULTS AND DISCUSSION

Across repeated demonstration trials covering panic-button activation, inactivity detection, sleep-mode suppression, warning cancellation, fall-like motion detection, and BLE-based caregiver notification, the prototype behaved correctly in approximately 90% of trials.

The panic-button, inactivity, sleep-mode, warning-cancellation, and BLE-notification pathways were consistently reliable. The occasional failures observed were concentrated in fall-like motion detection, consistent with the differing acceleration dynamics discussed in Section V-C: free-fall-style drops were detected more reliably than fast hand-held downward movement, since a held device is supported by the hand and therefore produces a noisier, less distinct acceleration signal than a released object in true free

fall. Tuning of the constants in Table II was used to balance sensitivity to genuine fall-like motion against rejection of shake- and tilt-only motion, and this trade-off is the main source of the remaining error margin.

X. LIMITATIONS

The prototype has several known limitations that should be considered when interpreting its results:

- The system is a university engineering prototype and is not a certified or clinically validated medical device; it should not be represented as providing clinical reliability.
- The MPU6050 cannot directly measure physical fall height; the nominal 1-meter requirement is approximated through motion duration, acceleration characteristics, downward movement, and impact/sudden-stop conditions, not a direct distance measurement.
- BLE communication range is limited; if a caregiver is outside BLE range, the direct BLE notification path will not reach them, and no long-range (Wi-Fi/cellular/cloud) notification is currently implemented.
- The current battery-level indication is a runtime-based estimate rather than a calibrated, directly measured voltage-based percentage.
- Fall-like motion detection accuracy using inertial measurements alone remains the primary technical limitation of the system, given the indirect and noisy nature of held-device acceleration signals.

XI. FUTURE WORK

Proposed future improvements include: a custom PCB and smaller enclosure; calibrated battery-voltage measurement using a voltage-divider circuit and the ESP32 ADC; sensor fusion to improve motion-state estimation; adaptive, context-aware detection thresholds; data-driven fall classification (e.g., using machine learning over a representative labelled dataset); GPS-based location reporting; Wi-Fi/cloud or GSM/cellular connectivity to provide long-distance caregiver notification beyond BLE range; a larger controlled test dataset; and more extensive user testing.

XII. CONCLUSION

This paper presented a compact, low-cost wearable elderly safety-band prototype that performs local emergency sensing and decision-making and provides caregiver notification through Bluetooth Low Energy. The system integrates motion sensing, manual panic activation, inactivity monitoring, local alerting, a warning/cancellation state machine, and a Flutter caregiver application, with all emergency decisions made locally on the ESP32-C3. Rather than competing with general-purpose commercial

smartwatches, the design is intentionally focused on elderly emergency response, emphasizing simple interaction, local alerts, and a staged, cancellable escalation path intended to reduce false emergency notifications. The prototype's fall-detection approach, its BLE-range limitation, and its runtime-based battery estimate are presented as explicit, acknowledged constraints of the current implementation rather than as fully solved problems, with concrete directions for future improvement identified in Section XI.

ACKNOWLEDGMENT

REFERENCES

- [1] Espressif Systems, “ESP32-C3 Series Datasheet,” Espressif Systems, 2023.
- [2] TDK InvenSense, “MPU-6000 and MPU-6050 Product Specification,” TDK InvenSense, Rev. 3.4.
- [3] Bluetooth SIG, “Bluetooth Core Specification,” Bluetooth Special Interest Group.
- [4] Google, “Flutter Documentation,” Google LLC. [Online]. Available: <https://docs.flutter.dev>
- [5] IEEE, “IEEEtran LaTeX Class Documentation,” IEEE, 2021.
- [6] Nanjing Top Power ASIC Corp., “TP4056 Li-ion Battery Charger IC Datasheet.”
- [7] Arosemi Technology, “MT3608 Step-Up Converter Datasheet.”